# CUSTOM API GATEWAY AND JWT AND RATE LIMITING

DEEPAK GANESH OSTWAL

GOVERNMENT COLLEGE OF ENGINEERING
CHHT.SAMBHAJINAGAR

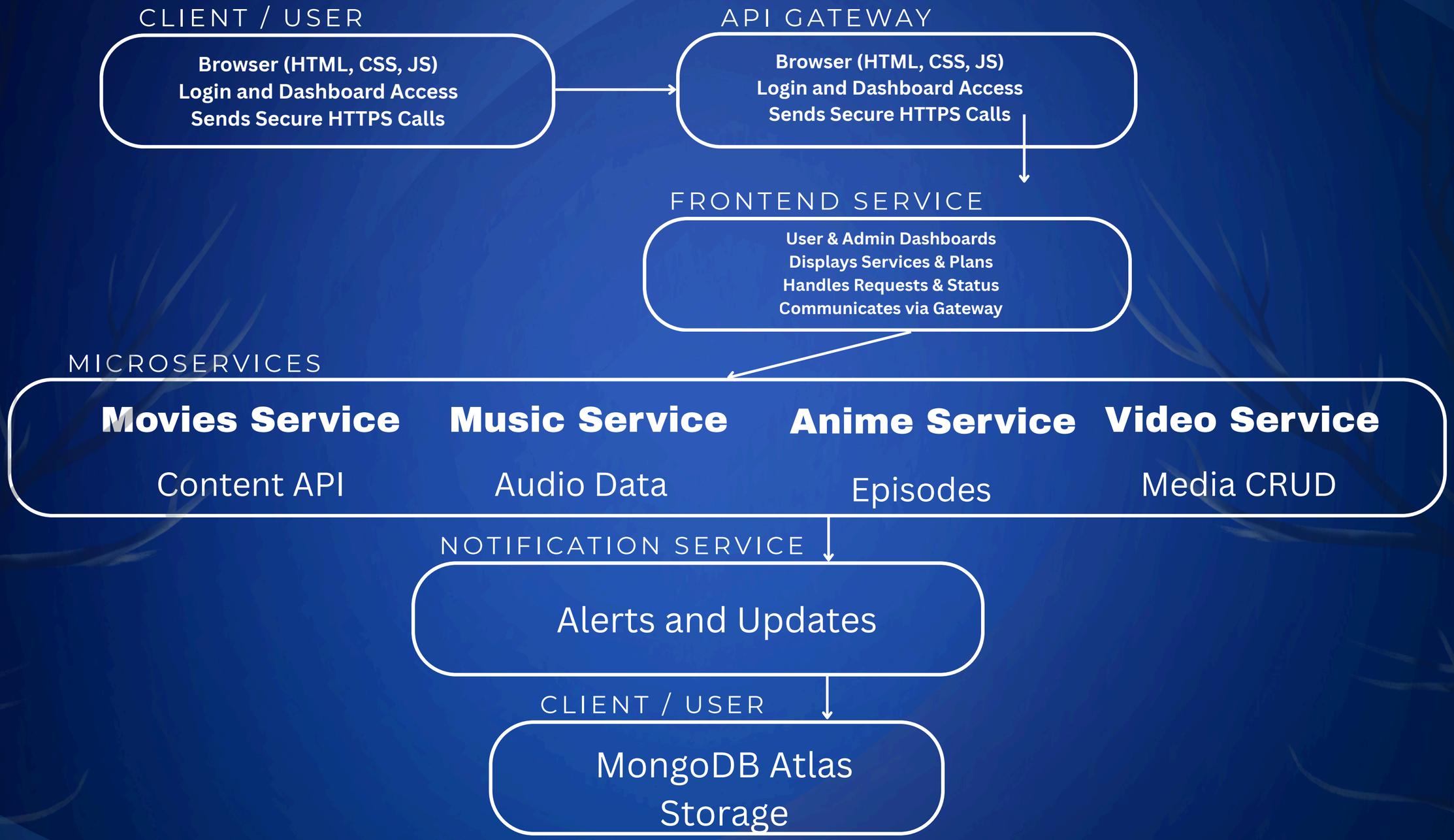
# PROJECT INFORMATION

This project is a Microservices-Based Platform designed around an API Gateway architecture that provides secure, scalable, and controlled access to multiple streaming services such as Movies, Music, Anime, Videos, and Notifications.
The platform includes centralized authentication, authorization, and user-role management, allowing both Admin and User functionalities under one integrated system.
It enables:

- Role-based Access Control (RBAC) through JWT authentication
- Centralized request routing and rate limiting via API Gateway
- Real-time notifications and user management
- Modular microservices connected through a unified data layer (MongoDB Atlas)

The entire system is built with Node.js, Express, and MongoDB, deployed using Kubernetes for scalability and resilience.

# SYSTEM ARCHITECTURE



# WORKING

## USERLOGIN

User opens the site, enters credentials, and logs in successfully.

## DASHBOARD ACCESS

Dashboard shows Movies, Music, Anime, Videos — available or locked.

## REQUEST OR PURCHASE

If a service is locked, user can request access or buy a plan (single or bundle).

## PAYMENT PROCESS

User completes payment securely, and access is unlocked instantly.

## ADMIN ACTION

Admin views all access requests and payments, then approves or manages permissions.

## SERVICE USAGE

User enjoys the unlocked services — streaming, viewing, or exploring content.

## NOTIFICATIONS

User and admin both receive real-time alerts for approvals, payments, and updates.



# DETAILED SYSTEM WORKING

**User Login**

- The user enters credentials on the frontend.
- The request is sent via API Gateway (Port 3000) to the Auth Service (3001).
- Auth Service verifies credentials using bcrypt and generates JWT tokens (Access + Refresh).
- Tokens and user data are returned to the frontend for authenticated sessions.

**Token Validation**

- For every future request, the JWT Access Token is attached to the header.
- API Gateway verifies the token before forwarding any request.
- Invalid or expired tokens are blocked immediately.

**Dashboard Access**

- The frontend fetches user details using /auth/me.
- Based on assigned permissions (stored in the token or DB), available services are displayed.
- Locked services are shown as “Request Access” or “Buy Plan.”

**Payment Flow**

- When the user buys a plan, a payment initiation request is sent to the Auth Service via the API Gateway.
- The Auth Service creates an order (mock or real, depending on setup) and verifies it.
- On success, it updates the user’s access list in MongoDB and unlocks the chosen services.
- A payment record is saved in the Payments collection.

**Access Request & Admin Approval**

- If the user requests access (instead of paying), an entry is created in accessRequests.
- Admin uses their dashboard to review pending requests.
- On approval, Auth Service updates the user’s assignedMicroservices array.
- A notification event is created for the user.

# DETAILED SYSTEM WORKING

**Access Request & Admin Approval**

- If the user requests access (instead of paying), an entry is created in accessRequests.
- Admin uses their dashboard to review pending requests.
- On approval, Auth Service updates the user's assignedMicroservices array.
- A notification event is created for the user.

**Service Interaction**

- Once access is granted, the user interacts with Movies, Music, Anime, and Video services.
- API Gateway routes the requests securely to the respective microservice (Ports 3003–3006).
- Each service communicates independently but reads from shared access data.

**Notification System**

- Every major event (login, access approval, payment success) creates a notification entry.
- Notification Service (Port 3007) handles delivery and retrieval.
- Users fetch notifications via /notifications/user, updated periodically.

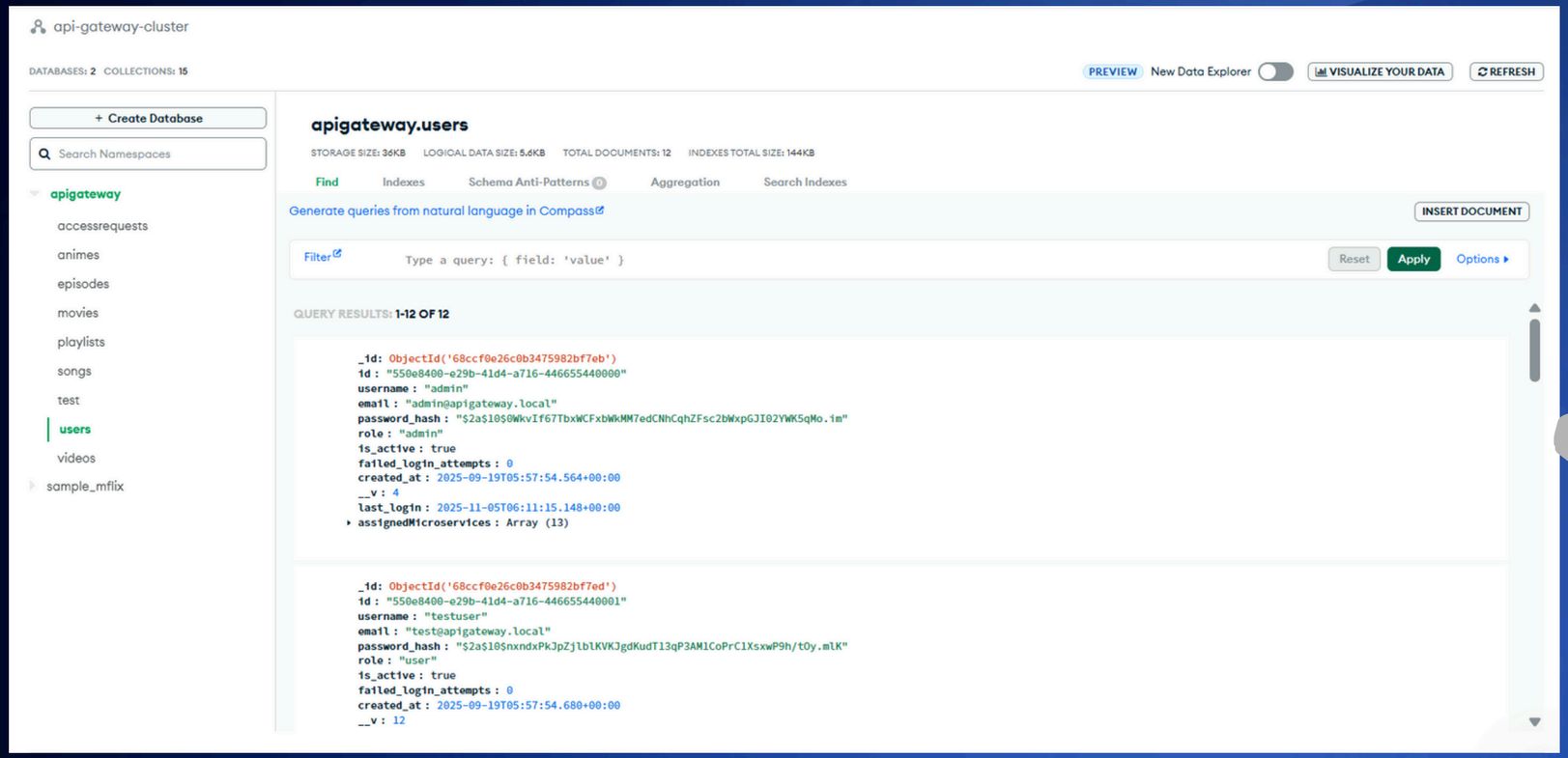
**Database Operations**

- All data (users, requests, payments, notifications) is stored in MongoDB Atlas.
- The system uses Mongoose ODM for CRUD operations.
- If MongoDB fails, global in-memory objects (global.users, global.notifications, etc.) ensure fallback continuity.

**Security & Reliability**

- JWT ensures stateless authentication (no session storage).
- Passwords are always hashed with bcrypt (10 rounds).
- API Gateway manages rate limiting, request logging, and CORS policies.
- Health checks monitor all microservices for uptime and quick recovery.

# GALLARY




The MongoDB Atlas cluster hosts collections for users, services, payments, and access requests.
Each user document includes encrypted credentials, JWT tokens, and permission arrays.
Mongoose ODM handles schema validation and CRUD operations.
Admin actions like access approval or payment updates modify these collections.
This design supports high availability and distributed scaling for all microservices

The API Gateway uses three independent rate limiters — general And authentication— to manage request flow efficiently.
Each limiter controls the number of allowed requests per minute for specific routes.
The general limiter restricts all routes to 1000 requests per minute per IP.
The authentication limiter prevents brute-force login attempts by allowing only 100 per minute.

api-gateway > JS server-simple.js > ...

```
1   const express = require('express');
2   const cors = require('cors');
3   const axios = require('axios');
4   const rateLimit = require('express-rate-limit');
5
6   const app = express();
7   const PORT = process.env.PORT || 3000;
8
9   // Rate limiting configurations
10  const generalLimiter = rateLimit({
11    windowMs: 1 * 60 * 1000,
12    max: 1000,
13    message: { error: 'Too many requests', message: 'Please try again later.' },
14    standardHeaders: true,
15    legacyHeaders: false,
16  });
17
18  const authLimiter = rateLimit({
19    windowMs: 1 * 60 * 1000,
20    max: 100,
21    message: 'Too many login attempts, please try again later.',
22    skipSuccessfulRequests: true,
23  });
24
25  const paymentLimiter = rateLimit({
26    windowMs: 1 * 60 * 1000,
27    max: 100,
28    message: { error: 'Too many payment requests', message: 'Please try again later.' },
29    standardHeaders: true,
30    legacyHeaders: false,
31  });
```

# TECHNICAL IMPLEMENTATIONS

## FRONTEND SERVICES

- **API Gateway:** Routing, auth, rate limiting, security
- **Auth Service:** Login, JWT, roles, bcrypt hashing
- **Microservices:** Movies, Music, Anime, Videos APIs
- **Notifications:** Alerts & updates
- **Database:** MongoDB Atlas (users, permissions, logs)
- **Payments:** Verification & access updates
- **Security & Deployment:** HTTPS, fallback, Docker, Kubernetes







## BACKEND SERVICES

- **Login & Dashboard:** Secure access, role-based views
- **Admin Panel:** Manage users, approvals, revenue
- **Payments:** Plan selection & confirmation
- **Notifications:** Alerts, messages, status
- **Storage:** JWT & user data in LocalStorage
- **Integration:** API calls with Bearer tokens
- **UI/UX:** Responsive, fast, user-friendly

# THANK-YOU